Programlama Laboratuvarı I Proje II
Savaş Araçları Kart Oyunu

Farahnozkhon Dilovarovna Masumova
Bilgisayar Mühendisliği
Kocaeli Üniversitesi
220201142

Nazire İpek Agan
Bilgisayar Mühendisliği
Kocaeli Üniversitesi
230201024

Özet– Nesne yönelimli programlama konseptleri kullanılarak oyuncunun ve bilgisayarın ellerindeki kartların dayanıklılık ve vuruş özellikleri kıyaslanmasıyla belirli hamleler sonucunda kazanının belirlendiği bir kart oyun tasarlandı. Java diliyle yazılan kodda nesne yönelimli programlamanın soyutlama, kalıtım, çok biçimlilik, kapsülleme gibi temel özellikleri kullanıldı. Oyunun aşamaları görselleştirilerek basit bir kullanıcı arayüzü tasarlandı.

I. Giriş

Java dilinde yazılan, kullanıcının bilgisayarla karşılıklı oynadığı bu savaş araçları kart oyununda kullanıcının başlangıçta altı kart seçmesi sağlanmıştır. Her bir kartın farklı nesneler olması için, altı tür kartın tutulduğu ArrayList yapısından farklı nesneler alınmıştır. Oyunun her hamlesinde oyuncu üç kart, bilgisayar da rastgele olarak üç kart seçmiştir. İki tarafın kartları sırayla karşılaştırılmış ve vuruş değeri, vuruş avantajı, dayanıklılık gibi değerlerle hesaplamalar yapılarak kartların yeni seviye puanları ve dayanıklılık değerleri hesaplanmıştır. Dayanıklılık değeri sıfıra ulaşan kartlar oyuncu ve bilgisayar listelerinden elenmiş ve her hamle sonunda iki tarafa da seviye puanları kontrol edilerek birer kart eklenmiştir. Bu şekilde ilerleyen oyunun sonunda hamle sayısı ya da skorlar karşılaştırılarak kazanan belirlenmiştir. Buna göre kazanan yazdırılmış ve oyuncuya bilgi verilmiş ve oyun sonlanmıştır.

II. Sınıfların Oluşturulması

SavasAraclari adlı abstract sınıfından extend edilerek Deniz, Hava ve Kara abtract sınıfları oluşturulmuştur. Bu üç sınıftan da ikişer adet extend edilerek oluşturulan Uçak, Siha, Fırkateyn, Sida, Obüs ve Kfs sınıfları, gerçek savaş araçları modellerine karşılık gelen nesneler oluşturabilmek için kullanılmıştır.

SavasAraclari sınıfı, üretilecek savaş araçları nesnelerinin temel özelliklerini ve davranışlarını tanımlayan soyut bir sınıftır. Bu temel özellikler *int dayanikilik*, *int seviyePuani*, *int vurus* ve *string sinif*'tir. *dayaniklilik*: Oluşturulacak her bir kartın dayanıklılık değerini tutar. s*eviyePuani*: Başlangıçta kullanıcının gireceği seviye puanı değerini tutar ve kartların yaptığı elemelere göre güncellenir. *vurus*: Kartların vuruş değerini tutar, her kart türü için farklıdır. *sinif*:. Kartların hangi sınıfa ait olduğunu tutar. Bu değişkenler protected olarak tanımlanmış ve nesne yönelimli programlamanın önemli bir konsepti olan kapsülleme projeye dahil edilmiştir. Protected değişkenler tercih edilmesinin nedeni, değişkenlerin class üzerinden extend edilen diğer classlar tarafından erişime açık olmasının sağlanmasıdır. Ancak public değişkenlerin aksine, tüm classlar tarafından erişime açık değildir, sadece extend edilen classlar ile kısıtlanmıştır. Bu da extend edilen classlar arasında iletişim için esneklik sağlarken, diğer classlardan yapılan yanlış müdahaleler engellenerek programın tutarlığını korumuştur.

Deniz sınıfı *SavasAraclari* sınıfından türetilmiş soyut bir sınıftır. Constructor içerisinde super() metodu ile *SavasAraclari* sınıfından *dayaniklilik*, *seviyePuani*, *vurus* ve *sinif* gibi temel özellikler alınmıştır. Ek olarak yine protected değişkenler olan *string altSinif* ve Deniz sınıfının Hava sınıfına karşı vuruş avantajı olduğundan *int havaVurusAvantaji* adlı değişkenler tanımlanmıştır.

Hava sınıfı da *Deniz* sınıfı gibi *SavasAraclari* sınıfından türetilmiş soyut bir sınıftır. Yine constructor içerisinde super() metodu ile

SavasAraclari sınıfının *dayaniklilik*, *seviyePuani*, *vurus* ve *sinif* gibi temel özellikleri alınmıştır. Ek olarak protected olan *string altSinif* ve Hava sınıfının Kara sınıfına karşı vuruş avantajı olduğu için *int karaVurusAvantaji* adlı değişkenler tanımlanmıştır.

Kara sınıfı da diğer iki sınıf gibi *SavasAraclari* sınıfından türetilmiş soyut bir sınıftır. Yine constructor içerisinde super() metodu ile *SavasAraclari* sınıfının *dayaniklilik*, *seviyePuani*, *vurus* ve *sinif* gibi temel özellikleri alınmıştır. Ek olarak protected olan *string altSinif* ve Kara sınıfının Deniz sınıfına karşı vuruş avantajı olduğu için *int denizVurusAvantaji* adlı değişkenler tanımlanmıştır.

Ucak sınıfı, *Hava* sınıfından türetilmiş bir sınıftır. Uçak nesnelerinin özelliklerini ve davranışlarını tanımlar. Hava sınıfından int dayaniklilik, int seviyePuani, int vurus, String sinif, String altSinif, int karaVurusAvantaji özelliklerini super() metodu ile miras alır. Ayrıca, getKartAdi metodu ile sınıfın kart adını "Ucak" olarak döndürür ve DurumGuncelle ile uçakların durumunu dinamik olarak günceller.

Firkateyn sınıfı, *Deniz* sınıfından türetilmiş bir sınıftır. Fırkateyn nesnelerinin özelliklerini ve davranışlarını tanımlar. Deniz sınıfından int dayaniklilik, int seviyePuani, int vurus, String sinif, String altSinif, int havaVurusAvantaji özelliklerini super() metodu ile miras alır. Ayrıca, getKartAdi metodu ile sınıfın kart adını "Firkateyn" olarak döndürür ve DurumGuncelle ile fırkateynlerin durumunu dinamik olarak günceller.

Obus sınıfı, *Kara* sınıfından türetilmiş bir sınıftır. Obüs nesnelerinin özelliklerini ve davranışlarını tanımlar. Kara sınıfından int dayaniklilik, int seviyePuani, int vurus, String sinif, String altSinif, int denizVurusAvantaji özelliklerini super() metodu ile miras alır. Ayrıca, getKartAdi metodu ile sınıfın kart adını "Obus" olarak döndürür ve DurumGuncelle ile obüslerin durumunu dinamik olarak günceller.

Siha sınıfı, *Hava* sınıfından türetilmiş bir sınıftır. Siha nesnelerinin özelliklerini ve davranışlarını tanımlar. Hava sınıfından int dayaniklilik, int seviyePuani, int vurus, String sinif, String altSinif, int karaVurusAvantaji özelliklerini super() metodu ile miras alır. Ayrıca, overirde edilen getKartAdi metodu ile sınıfın kart adını "Siha" olarak döndürür ve DurumGuncelle ile sihaların durumunu dinamik olarak günceller. Aynı zamanda public int getSihaDenizVurusAvantaji() ile Siha türünün Deniz altındaki class nesnelerine karşı olan avantajını dönüdürür.

Sida sınıfı, *Deniz* sınıfından türetilmiş bir sınıftır. Sida nesnelerinin özelliklerini ve davranışlarını tanımlar. Deniz sınıfından int dayaniklilik, int seviyePuani, int vurus, String sinif, String altSinif, int havaVurusAvantaji özelliklerini super() metodu ile miras alır. Ayrıca, override edilen getKartAdi metodu ile sınıfın kart adını "Siha" olarak döndürür ve DurumGuncelle ile sidaların durumunu dinamik olarak günceller. Aynı zamanda public int getSidaKaraVurusAvantaji() ile Siha türünün Kara altındaki class nesnelerine karşı olan avantajını dönüdürür.

Kfs sınıfı, *Kara* sınıfından türetilmiş bir sınıftır. Kfs nesnelerinin özelliklerini ve davranışlarını tanımlar. Kara sınıfından int dayaniklilik, int seviyePuani, int vurus, String sinif, String altSinif, int denizVurusAvantaji özelliklerini super() metodu ile miras alır. Ayrıca, override edilen getKartAdi metodu ile sınıfın kart adını "Siha" olarak döndürür ve DurumGuncelle ile KFS durumlarını dinamik olarak günceller. Aynı zamanda public int getKfsHavaVurusAvantaji() ile Kfs türünün Hava altındaki class nesnelerine karşı olan avantajını dönüdürür.

Oyuncu sınıfı, oyundaki oyuncuları temsil eden sınıftır. Protected değişkenler olan int oyuncuID, String oyuncuAdi, int skor, ArrayList<SavasAraclari> kartListesi ve private değişken olan int baslangicSeviyePuani değişkenlerinden oluşmaktadır. Public void skorGoster() fonksiyonu ile oyuncuların skoru gösterilir, public void kartEkle(SavasAraclari kart) fonksiyonu ile oyuncuların kartlarının tutulduğu listeye kartlar eklenir, public void skorEkle(int skor) ile oyuncuların skorları oyun esnasında güncellenir. Public void setBaslangicSeviyePuani(int) ile başlangıçta kullanıcının girdiği seviye puanı değeri, oyuncunun listesindeki tüm kartların seviyePuani değişkenine atanır. Public int getBaslangicPuani() fonksiyonu da bir getter fonksiyonu olarak çalışır ve oyunda kullanılmak üzere private int

baslangicSeviyePuani değerini döndürür. Bu sınıftan kullanıcıyı temsil etmek üzere bir oyuncu nesnesi, bilgisayarı temsil etmek üzere de bir bilgisayar nesnesi oluşturulur ve oyun boyuncu bu nesnelerin özellikleri karşılaştırılır, hesaplamalar ve güncellemeler yapılır.

Oyun sınıfı, tüm oyunu temsil eden sınıftır ve main() fonksiyonu da bu sınıf içerisinde bulunur. Oyunun başında bu sınıftan oluşturulan nesne ile oyun başlatılır ve yine aynı sınıf içindeki işlemler ile kazanan belirlenir. Oyun sınıfı Oyuncu türünde bir protected oyuncu değişkenine, Oyuncu türünde bir protected bilgisayar değişkenine, oyun boyunca tüm savaş kartlarının tutulduğu ArrayList<SavasAraclari> tumKartlar adlı protected bir array listeye sahiptir. public void KartlariHazirla() adlı, tüm kart seçeneklerini tutan tumKartlar listesini hazırlamak için oluşturulmuş bir fonksiyondur. Public void KartDagit() fonksiyonu ise oyuncunun başlangıç seviye puanını kontrol ederek oyunculara ilk altı kartı dağıtan fonksiyondur. Bu ilk altı kartı oyuncu kendi seçerken bilgisayar ise rastgele alır. Oyun sınıfı içinde publiic void oyunBaslat(), public int SaldiriHesapla(SavasAraclari saldiran, SavasAraclari digerTaraf), public ArrayList<SavasAraclari> kartSec(boolean isBilgisayar, List<SavasAraclari> musaitKartlar, List<SavasAraclari> oncedenSecilenKartlar) fonksiyonları ise oyun süresince hesaplama yapmak için kullanılan fonksiyonlardır.

Oyun sınıfının constructor'ında çağırılan kartlariHazirla () fonksiyonu ile tumKartlar listesine her araç türünden bir nesne yerleştirilir. Bu, add() fonksiyonu ile oluşturulan nesnelerin listeye eklenmesiyle gerçekleşir. Örneği uçak nesnesi oluşturulurken Ucak sınıfının dayanıklılık, başlangıç seviye puanı, vuruş değeri, sınıfı, alt sınıfı, kara vuruş avantajı değerleri girilerek nesneler oluşturulmuştur. Bu listede nesnelerin sırası önemlidir, Ucak, Obus, Firkateyn, Siha, Sida ve Kfs şeklindedir çünkü oyuncunun seviyesinin 20'nin altında olması durumunda sadece ilk üç elemandan, daha fazla olması durumunda ise tüm altı elemandan seçim yapması sağlanacaktır.

Listedeki nesnelerin seçilip oyuncu.kartListesi ya da bilgisayar.kartListesi listelerine eklenmesi sırasında, her adımda tumKartlar listesi clear() fonksiyonu ile temizlenmiş, sonrasında KartlariHazirla() fonksiyonu çağırılması ile yeniden oluşturulmuştur. Böylece her seçimde, oyuncu aynı indeksi seçse dahi, farklı nesnelerin listeye eklenmesi sağlanmıştır.

III. Oyunun Başlatılması

main() metodunda Oyuncu sınıfından oyuncu ve bilgisayar adlı başlangıç skorları 0 olan iki nesne oluşturulur. Kullanıcıdan seviye puanı değerini girmesi istenir, seviye puanı değerinin sıfırdan büyük olup olmadığı kontrol edilir ve oyuncu ve bilgisayar nesnelerinin baslangicSeviyePuani değerleri setter fonksiyonu ile kullanıcının girdiği değere eşitlenir. Daha sonra kullanıcıdan hamle sayısı girmesi istenir ve girilen bu sayıya göre Oyun sınıfından oyun adlı bir nesne oluşturulur. Oyuncu ve bilgisayar nesneleri ile hamle sayısı bu oyun nesnesinin özellikleridir. Daha sonra oyunBaslat() fonksiyonu çağırılır ve oyun başlatılmış olur.

Public void oyunBaslat() fonksiyonu, oyunun hamlelerinin yürütüldüğü fonksiyondur ve kazanan da burada belirlenir.

IV. Hamlede Kart Seçimi

Öncelikle kartlariHazirla() fonksiyonu ile tumKartlar listesi oluşturulur, bu listeden oyuncunun altı kart seçmesi ve bilgisayara rastgele altı kart verilmesi için kartDagit() fonksiyonu çalışır. Seviye puanları kontrol ederek oyunculara ilgili kartlar altışar adet dağıtılır. Her kart verilişinden sonra tumKartlar listesi clear ile temizlenir ve kartlariHazirla() fonksiyonu ile tekrar oluştulur ve bu sayede oyuncunun aynı türde nesneyi alsa bile, mesela iki kez aynı 0. indeksteki uçağı alsa bile, farklı nesneler alması sağlanmıştır. Bu hesaplama kısmında nesnelerin özellikleri üzerinde değişiklikler yapılacağı için önem arz etmektedir.

Bilgisayar için previouslyChosenCardsBilgisayar, oyuncu için ise previouslyChosenCardsOyuncu listeleri oluşturulur. Bu listelerde, hamlelerde oyuncuların daha önce seçtiği kartları tekrar seçmeden önce, seçilmeyen tüm kartları seçmesi kuralı uygulamak için, seçilen kartlar tutulacaktır.

Oyunun hamle sayısı sınır olarak bir for döngüsü başlatılır. public ArrayList<SavasAraclari > kartSec(boolean isBilgisayar, List <SavasAraclari> musaitKartlar, List <SavasAraclari> oncedenSecilenKartlar) fonksiyonuna, oyuncu için false, oyuncu.kartListesi ve previouslyChosenCardsOyuncu parametre olarak gönderilir. Bilgisayar içinse true değeri ve bilgisayara ait listeler gönderilir. Bu fonksiyonda öncelikle boolean değeri kontrol edilir ve true ise bilgisayar için işlem yapılır. Bilgisayar hamle için random üç kart seçecektir, bu yüzden önce bilgisayarIcinMusait listesi oluşturulur ve oncedenSecilenKartlarBilgisayar listesi buraya kopyalanır.
bilgisayarIcinMusait.removeAll(oncedenSecilenKartlar); komutu ile müsait kartlar listesinden önceden seçilen kartlar temizlenir. Int türündeki secilmeyenSayisiDegiskeni, bilgisayarIcinMusait listesinin boyutuna eşitlenir ve kontrol yapılarıyla değerine göre işlem yapılır. Eğer 2'den büyükse sadece seçilmeyen kartlar arasından seçim yapılır, eğer 2 ise iki adet seçilmeyenlerden seçilir ve bir adet daha önce seçilenlerden seçilir, eğer 1 ise seçilmeyen bir adet kart seçilir ve daha önce seçilenlerden iki adet seçilir. Else, yani seçilmeyen kart sayısı 0 ise, tüm seçimler daha önce seçilmiş kartlar arasından yapılır. Tüm bu seçimler random bir indeks değeri oluşturularak yapılır ve fonksiyon başında oluşturulan secilenKartlar listesine kaydedilir. Bu liste, hamle boyunca elde edilen kartları tutar ve her hamlede döndürülür.

Eğer kartSec fonksiyonuna gönderilen Boolean değeri False ise oyuncu seçim yapar. Bilgisayarın rastgele kart seçimiyle aynı mantık devam eder ancak oyuncunu kartları rastgele seçilmez, kullanıcı kendi seçer.

Bu fonskiyondan dönen listeler oyuncuSecilenKartlar ve bilgisayarSecilenKartlar listelerine kopyalanır. Daha sonra bu listelerin 0 ile 0, 1 ile 1 ve 2 ile 2. indeksindeki kartlar karşılaştırılır ve hamle bitmiş olur.

V. Hamle Sırasında Hesaplama

Bu aşamda, oyuncuSecilenKartlar ve bilgisayarSecilerKarlar listelerinin boyutu her adımda 3 olacağından, 3 sınırlı bir for döngüsü başlatılmıştır. Burada iki tarafın elemanları sırayla karşılaştırılmıştır. KartPuaniGoster() metodu ile kartların özellikleri yazdırılmış ve daha sonra public int SaldiriHesapla(SavasAraclari, SavasAraclari) fonksiyonuna sırasıyla oyuncu ve bilgisayar saldıran ve savunan olarak gönderilmiştir. SaldırıHesapla fonksiyonu, tarafların vuruş avantajlarını kontrol ederek ve toplayarak vuruş değerini döndürür. Saldıran taraf oyuncuysa, bilgisayarın da türüne göre vuruş değeri hesaplanır ve vurusOyuncu değişkenine kaydedilir, aynı şekilde saldıran bilgisayar olduğunda vuruş değeri vurusBilgisayar değişkenine kaydedilir. Bu fonksiyon temelde iki tarafın türlerinin birbirine karşı avantaşı olup olmadığını ve Siha, Sida ve Kfs gibi türlerin özel avantajları olup olmadığını kontrol eder. Eğer varsa getSihaDenizVurusAvantaji() gibi metotlar ile bu değerlere ulaşır ve nesnenin toplam vuruş değerine ekler. Kontrollerden sonra da toplam vuruş değerini döndürür.
Döngüye geri dönecek olursak, nesnelerin dayanıklılık değerinden karşı tarafın vuruş değeri çıkartılır ve nesnenin yeni dayanıklılık değeri olarak güncellenir. Eğer yeni dayanıklılık 0'dan büyük ise kart oyuna devam eder ama 0'dan küçükse kart elenir ve karşı tarafa kartın seviye puanı kadar skor eklenir. Tüm hamle için iterasyonda gerekli kıyaslamalar yapıldıktan sonra dayanıklılık değeri kontrol edilerek kartı listeden silme işlemi yapılır. Oyuncu.kartListesi ve bilgisayar.kartListesi döngüyle taranır ve içinde dayanıklılık değeri 0 olan kart bulursa listeden çıkarılır. Listenin boyutu bir azalacağından, iterasyon için kullanılan değişkenin de değeri bir azaltılır ve döngü devam eder.
Hamle içindeki kıyaslamalar ve kart elemeleri bittikten sonra her iki tarafa da duruma göre bir ya da iki kart eklenmesi gerekmektedir. Bunun için oyuncuların kartListesi listelerinin boyutları kontrol edilir. İki ya da ikiden büyükse bir kart, bir ise iki kart eklenir. Kart eklemeden önce oyuncuların puanları da kontrol edilir ve 20'den küçükse sadece Ucak, Obus ve Firkateyn nesneleri eklenebilirken 20 ve üstü olduğundan her sınıfın nesnesi eklenebilir.

Hamlelerin döngüsü bittikten sonra kazanan belirlenir. Bunun için önce oyuncuların skorları karşılaştırılır. Eğer skorlar eşitse toplam dayanıklılık puanları karşılaştırılır ve fazla olan kazanır. Eğer toplam dayanıklılık değerleri de eşitse oyun berabere biter.

V. DENEYSEL SONUÇLAR

Oyun boyunca yapılan hesaplamalar, hamle içi adımlar, eklenen ve silinen kartlar yazdırılarak oyunun çalışması kontrol edilmiştir. Oyun kuralları gereği oyuncunun bilgisayarın kartlarını görmemesi gerekmektedir ama sonuçların doğruluğunu kıyaslayabilmek için bilgisayarın elindeki kartlar da yazdırılmış ve oyunun gidişatı takip edilmiştir.

```
Seviye sayisi giriniz:
10
Hamle sayisi giriniz:
2
Lutfen kart seciniz (0, 1 veya 2):
0
0: Ucak
Lutfen kart seciniz (0, 1 veya 2):
0
1: Ucak
Lutfen kart seciniz (0, 1 veya 2):
1
2: Obus
Lutfen kart seciniz (0, 1 veya 2):
2
3: Firkateyn
Lutfen kart seciniz (0, 1 veya 2):
2
4: Firkateyn
Lutfen kart seciniz (0, 1 veya 2):
1
5: Obus

Oyuncu:
0: Ucak
1: Ucak
2: Obus
3: Firkateyn
4: Firkateyn
5: Obus

Bilgisayar:
1: Ucak
2: Obus
3: Ucak
4: Obus
5: Ucak
6: Ucak
```

```
Hamle 1
Lutfen kart seciniz (0 ile 5 arasinda sayilar):
0
Secilen kart: Ucak
Lutfen kart seciniz (0 ile 5 arasinda sayilar):
1
Secilen kart: Ucak
Lutfen kart seciniz (0 ile 5 arasinda sayilar):
2
Secilen kart: Obus
1. hamle basinda Oyuncu kullanicisinin sectigi kartlar:
Ucak
Ucak
Obus
1. hamle basinda Bilgisayar kullanicisinin sectigi kartlar:
Ucak
Obus
Ucak

 ------ Ucak VS Ucak ------

 Oyuncu
 Karti: Ucak
 Dayaniklilik: 20
 Seviye Puani: 10
 Vurus: 10

 Bilgisayar
 Karti: Ucak
 Dayaniklilik: 20
 Seviye Puani: 10
 Vurus: 10


 Oyuncu kullanicisinin Ucak kartinin toplam vurus degeri: 10

 Bilgisayar kullanicisinin Ucak kartinin toplam vurus degeri: 10

 Ucak kartinin yeni dayaniklilik degeri: 10

 Ucak kartinin yeni dayaniklilik degeri: 10

 ------ Ucak VS Obus ------

 Oyuncu
 Karti: Ucak
 Dayaniklilik: 20
 Seviye Puani: 10
 Vurus: 10

 Bilgisayar
 Karti: Obus
 Dayaniklilik: 20
 Seviye Puani: 10
 Vurus: 10

 Ucak kartinin Obus kartina karsi 10 kadar Kara vurus avantaji var!

 Oyuncu kullanicisinin Ucak kartinin toplam vurus degeri: 20

 Bilgisayar kullanicisinin Obus kartinin toplam vurus degeri: 10

 Obus kartinin yeni dayaniklilik degeri: 0

 Ucak kartinin yeni dayaniklilik degeri: 10

 Obus elendi!

 Oyuncu kullanicisinin skoru 10 artti ve yeni skoru 10 oldu.
```

```
------ Obus VS Ucak ------

Oyuncu
Karti: Obus
Dayaniklilik: 20
Seviye Puani: 10
Vurus: 10

Bilgisayar
Karti: Ucak
Dayaniklilik: 20
Seviye Puani: 10
Vurus: 10

Ucak kartinin Obus kartina karsi 10 kadar Kara vurus avantaji var!

Oyuncu kullanicisinin Obus kartinin toplam vurus degeri: 10

Bilgisayar kullanicisinin Ucak kartinin toplam vurus degeri: 20

Ucak kartinin yeni dayaniklilik degeri: 10

Obus kartinin yeni dayaniklilik degeri: 0

Obus karti elendi!

Bilgisayar kullanicisinin skoru 10 artti ve yeni skoru 10 oldu.
```

Görsellerde görüldüğü gibi her hamlede üçer kart seçilmiş ve bu kartlar karşılaştırılmıştır. Başlangıçta seviye puanı 20'den küçük olduğu için seçilecek ilk altı kart sınırlandırılmıştır.

```
Bilgisayar kullanicisinin skoru 10 artti ve yeni skoru 10 oldu.

Puanlar hesaplandiktan ve elemelerden sonra kart listesi (Oyuncu)
0. Ucak
1. Ucak
2. Firkateyn
3. Firkateyn
4. Obus

Puanlar hesaplandiktan ve elemelerden sonra kart listesi (Bilgisayar)
0. Ucak
1. Ucak
2. Obus
3. Ucak
4. Ucak

Hamle sonu Oyuncu skoru: 10

Oyuncu kullanicisina 1 adet Ucak karti eklendi.

0: Ucak
1: Ucak
2: Firkateyn
3: Firkateyn
4: Obus
5: Ucak

Hamle sonu Bilgisayar skoru: 10

Bilgisayara 1 adet kart eklendi.

0: Ucak
1: Ucak
2: Obus
3: Ucak
4: Ucak
5: Obus
```

Aynı şekilde hamle sonu skorlar ve elemelerden sonra elde kalan kartlar yazdırılarak kontrol sağlanmıştır.

```
Hamle 2
Lutfen kart seciniz (0 ile 5 arasinda sayilar):
1
Bu kart zaten secildi! Lutfen baska bir kart secin.
Lutfen kart seciniz (0 ile 5 arasinda sayilar):
2
Secilen kart: Firkateyn
Lutfen kart seciniz (0 ile 5 arasinda sayilar):
0
Bu kart zaten secildi! Lutfen baska bir kart secin.
Lutfen kart seciniz (0 ile 5 arasinda sayilar):
3
Secilen kart: Firkateyn
Lutfen kart seciniz (0 ile 5 arasinda sayilar):
4
Secilen kart: Obus
2. hamle basinda Oyuncu kullanicisinin sectigi kartlar:
Firkateyn
Firkateyn
Obus
2. hamle basinda Bilgisayar kullanicisinin sectigi kartlar:
Ucak
Obus
Ucak
```

Görselde de görüldüğü gibi ikinci hamle için kart seçme sırasında kartların daha önce seçilip seçilmediği kontrol edilmiştir. Eğer daha önce seçilmişse "Bu kart zaten seçildi! Lütfen başka bir kart seçin." uyarısı vererek kullanıcıdan yeni bir kart seçmesi istenmiştir. Bunun gibi birçok denemeyle kurulan sistemlerin doğru çalıştığı teyit edilmiştir.

VI. GÖRSELLEŞTİRME

Oyunun görselleştirme kısmı JavaFx kullanarak yapılmıştır. JavaFX, Java ile masaüstü uygulamaları geliştirmek için kullanılan bir grafik kullanıcı arayüzü (GUI) framework'üdür ve zengin görseller, animasyonlar ve etkileşimli arayüzler oluşturmak için çeşitli araçlar sunar.

Bu oyunda da JavaFx'in sunduğu çeşitli kütüphaneler kullanılmıştır.
javafx.application.Application
JavaFX uygulamalarını başlatmak için temel sınıftır.
Javafx.scene.Scene, bir JavaFX uygulamasındaki görünüm içeriğini temsil eder. Sahneye eklenen her şey, pencerenin kullanıcıya gösterilen kısmıdır.
Javafx.stage.Stage ise JavaFX'te bir pencereyi temsil eder. Stage birinci pencere veya yeni bir pencere olabilir.
Oyunun başında kullanıcıya seviye sayısı ve hamle sayısını girmesini isteyen iki ekran sunulur. İlk Sahne (ilkSahne Metodu), kullanıcıdan seviye sayısını girmesini isteyen bir arayüz oluşturmuştur. İlk ekranda Seviye Puanı girilmesi istenir ve kullanıcının girdiği sayıya göre oyundaki kartların seviye puanı belirlenmiş olur. Aynı zamanda ikinciSahneMetodu ile yine kullanıcının girdiği hamle sayısı ile oyunun kaç

hamle süreceği belirlenir. Kullanıcı sayıları girdikten sonra Göder butonuna basar ve değerler alınmış olur. Veri girişi yapan butonlara basıldığında veri alınması submitButton.setOnAction (...) fonksiyonu içinde yapılmıştır.

Daha sonra üçüncü ekrana geçilir ucuncuSahneMetodu ile kullanıcı 6 kart seçer. Eğer girilen seviye puanı 20'den az ise sadece Uçak, Obüs ve Fırkateyn kartlarından seçim yapabilir ancak 20'den yüksekse seçime tüm dahil edilir. Dördüncü ekranda ise oyuncu kendi seçtiği altı kartı görecektir. Aynı zamanda bilgisayarın rastgele seçtiği altı kart da soru işaretleriyle temsil edilmiştir.

VII. UML DİYAGRAMI

Her oyuncunun elinde bir kart listesi bulunduğundan ve bu kartlar savaş araçlarından oluştuğundan Oyuncu -> SavasAraclari arasındaki ilişki composition olarak belirlenmiştir. Yani oyuncu olmadan bu kartlar yok olur.

Oyun sınıfı, iki oyuncuyu ve savaş araçlarını yönetiyor, hamle sayısını takip ediyor ve saldırı hesaplamaları içeriyor, bu yüzden Oyun -> Oyuncu arasındaki ilişki Aggregation olarak belirlenmiştir. Oyun sınıfı iki oyuncuyu içerebilir ama Oyun sınıfı olmasa dahi Oyuncu sınıfı nesneleri var olabilir.

SavasAraclari bir üst sınıf (abstract superclass) olarak, tüm savaş araçlarının ortak özelliklerini ve davranışlarını tanımlar. Bunlar, dayaniklilik, vurus, seviyePuani, sinif gibi ortak özelliklerin ve KartPuaniGoster gibi ortak metodlardır. Yani SavasAraclari ve Deniz, Hava, Kara arasında kalıtım ilişkisi vardır.
Kara, Deniz ve Hava'dan türetilen ikişer tane olmak üzere diğer altı sınıfla da aralarında kalıtım ilişkisi vardır.

```
Oyun "1" --> "2" Oyuncu : oyuncular
Oyuncu "1" *--> "1..*" SavasAraclari : kartlar
 SavasAraclari <|-- Hava
 SavasAraclari <|-- Deniz
 SavasAraclari <|-- Kara
 Hava <|-- Ucak
 Hava <|-- Siha
 Deniz <|-- Fırkateyn
 Deniz <|-- Sida
 Kara <|-- Obus
 Kara <|-- Kfs
```

UML diyagramı:

(Boyutu ayarlanamadığı için png dosyaya eklenmiştir.)

VIII. SONUÇ

Bu savaş araçları kart oyunu projesi, soyutlama, kalıtım, çok biçimlilik, kapsülleme gibi nesne yönelimli programlama konseptlerini başarıyla uygulan bir Java programıdır. Projede abstract türünde class oluşturma, bu classtan türetilen classlarda fonksiyonları override etme, bu classlardan türetilen class nesnelerinin özelliklerini gereken yerlerde kullanıp hesaplama yapma, nesne özelliklerini güncelleme, nesneler için sınıf fonksiyonlarını çağırma, listeleri etkili bir biçimde kullanma gibi birçok adımı içermektedir. İki tarafın kartlarını karşılaştırıp birçok veriyi kontrol ederek kart silip ekleyerek listeleri güncelleyen ve her hamlede bunu gerçekleştiren bu program, görselleştirme aracılığıyla kullanıcıya rahat bir deneyim sunmaktadır. Nesne yönelimli programlama ve konseptleriyle yeni tanışmış öğrenciler için Java dilinin geniş imkanlarını kullanma şansı sağlandığından önemli ve öğretici bir proje olmuştur.

KAYNAKLAR

[1] "Java abstraction". Bro Code. 6 Ekim 2020. https://www.youtube.com/watchv=Lvnb83qt57g

[2] "Java access modifiers: (public, protected, private)" . Bro Code. 6 Ekim. 2020. https://www.youtube.com/watch?v=T632kAJ_9VA

[3] "Java super keyword". W3schools. Erişim Tarihi: 28 Kasım 2024.

https://www.w3schools.com/java/ref_keyword_super.asp
[4] "When do I use super()?". Stackoverflow. 2017. https://stackoverflow.com/questions/4090834/when-do-i-use-super
[5] "UML Class Diyagramları". Tuğrul Bayrak. 24 Mayıs 2019. https://tugrulbayrak.medium.com/uml-class-diyagramlari-4c3bb7e9cc4c
[6] "UML Class Diagram." . JavaPoint. Erişim Tarihi: 30 Kasım 2024. https://www.javatpoint.com/uml-class-diagram
[7] "UML Class Diyagram Çizimi.". sina5an. 21 Kasım 2023. https://youtube.com/watch?v=G_V9nlvJ44I
[8] "JavaFx" . Bro Code. 17 Kasım 2020. https://www.youtube.com/playlist?list=PLZPZq0r_RZOM-8vJA3NQFZB7JroDcMwev
[9] "JavaFx Tutorial for Beginners.". Programming Knowledge. 18 Ekim 2015. https://www.youtube.com/playlist?list=PLS1QulWo1RIaUGP446_pWLgTZPiFizEMq
[10] "How To Setup JavaFX 21 and JDK 21 on Netbeans IDE//how to setup javafx 21 and jdk 21 on netbeans ide". Tech Codism with Farhana. 27 Kasım 2023. https://www.youtube.com/watch?v=Iun8wEtn4Zs
[11] Image1.png : https://www.pngwing.com/ru/free-png-zmhrh
[12] Image2.png: https://ar.culture.ru/ru/subject/sredniy-tank-t-44
[13] Image3.png: http://www.almaz-kb.ru/products/voennogo-naznacheniya/korabli-protivominnoy-oborony/protivominnyy-korabl-bazovoy-zony-proekta-12701/
[14] Image4.png: https://lenta.ru/articles/2022/12/24/bpla/
[15] Image5.png: https://ru.lovepik.com/image-380569645/lorry-clipart-colorful-drawing-of-a-truck-with-a-dumper.html
[16] Image6.png: https://www.google.com/imgres?imgurl=https://img.piri.net/mnresize/720/-/resim/upload/2022/08/01/09/49/f77d01b74.jpg&tbnid=oqBSyFkpm4czxM&vet=1&imgrefurl=https://www.yenisafak.com/foto-galeri/ekonomi/sida-nedir-sidanin-ozellikleri-neler-2067339&docid=N5EWHjR1kzyHxM&w=720&h=479&hl=ru-tj&source=sh/x/im/m5/3&kgs=c09796adfda13d68&shem=abme,trie